CALIAN.AI

Integrated Architecture & Step-by-Step Application Startup Guide

Angular Electron Desktop | Ollama Llama 3.2 | Python A2A | Paradigm AI Service | RabbitMQ

This guide integrates the solution architecture directly into the startup procedure. The demonstrated registry contains **seven agents**: one local North Agent and six agents discovered from the Paradigm AI Service through RabbitMQ. Ollama performs intelligent agent selection. Agent execution requires separate verification.

Architecture overview — understand the components before startup

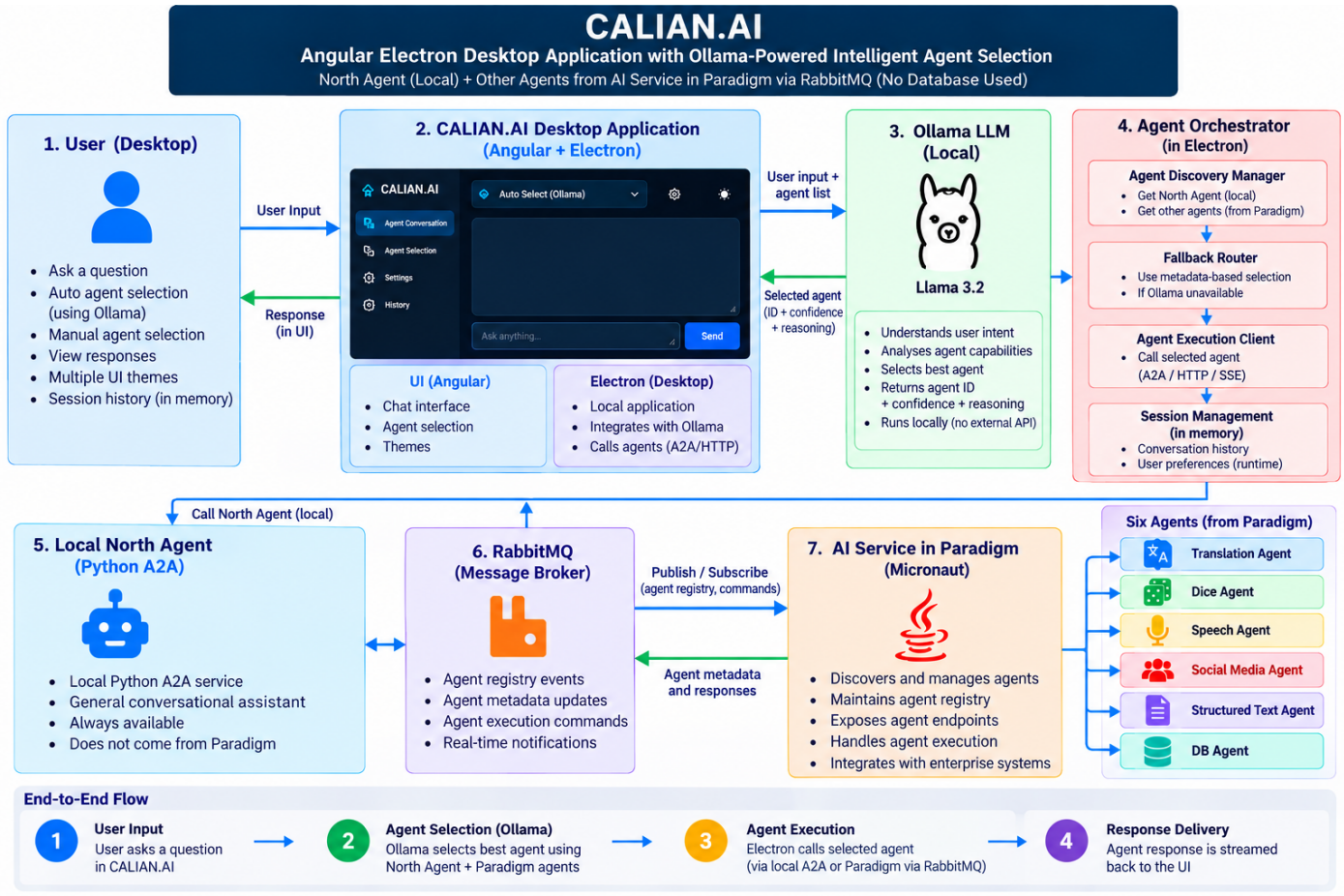


Figure 1. CALIAN.AI architecture: Angular Electron, local Ollama, local North Agent, RabbitMQ, Paradigm AI Service, and six Paradigm agents. No application database was demonstrated.

Reading the architecture: The desktop UI gathers the request; the Electron orchestrator unifies agent discovery; Ollama selects the best agent. The North Agent is local. The six other agents are supplied by the Paradigm AI Service through RabbitMQ. Agent execution is separate from selection.

1. Prerequisites

Component	Purpose
Windows 10/11	Desktop environment
Node.js and npm	Angular/Electron application
Python 3	Python router and local A2A service
Ollama + llama3.2	Local intelligent agent selection
RabbitMQ	Paradigm messaging
Paradigm AI Service (Micronaut)	Discovery of six enterprise agents
North Agent	Local Python A2A agent

2. Prepare the application

Extract the working application ZIP into a separate folder and retain existing backups. Locate the Electron project and the Python backend. Folder and launcher names may vary by distribution. Do not change existing working service settings.

```
CALIAN_AI_ELECTRON_THEMES/
  package.json
  electron/
  server/
  src/
PYTHON_BACKEND/
  main.py
```

3. Start Ollama (architecture component 3)

Open Command Prompt and check that the local Llama 3.2 model is installed. Start the Ollama server only if it is not already running.

```
ollama list
ollama pull llama3.2   (only if missing)
ollama serve          (only if not already running)
```

Ollama normally listens at **127.0.0.1:11434**. In another terminal, verify the model:

```
ollama run llama3.2 "Say hello"
```

4. Start the Python Ollama router

If present, double-click **START-PYTHON-ROUTER.bat**. Otherwise open a terminal in the Python backend folder, install the project dependencies, and run:

```
python -m uvicorn main:app --host 127.0.0.1 --port 9999
```

Open **http://127.0.0.1:9999/docs** and confirm **POST /router/select** is listed. This selection router does not replace the North Agent execution service.

5. Start the local North Agent (architecture component 5)

Start the existing Python A2A North Agent using its established startup procedure. Use its actual configured host and port, separate from the Ollama router. Do not guess or change a working port.

```
Ollama LLM:      127.0.0.1:11434
```

```
Ollama router:   127.0.0.1:9999
North A2A agent: <existing host and port>
```

6. Start RabbitMQ and Paradigm AI Service (components 6–7)

Start or verify the existing RabbitMQ broker, followed by the Paradigm Micronaut AI Service. These components supply the six non-North agents via the established messaging and registry workflow. Their installation and startup procedures are environment-specific.

Paradigm agent	Function
Translation Agent	Language translation
Dice Agent	Dice rolling and random results
Speech Agent	Speech operations
Social Media Agent	Content generation
Structured Text Agent	Text extraction and structuring
DB Agent	Database-related operations

Note: The DB Agent is one of the discovered agents; it does not establish that the CALIAN.AI desktop application itself uses a database. No application database interaction was demonstrated.

7. Check service configuration

Preserve the existing Paradigm settings. Verify that the desktop application targets the actual North A2A endpoint, the Ollama router, and the Paradigm integration. The following are illustrative setting names, not verified configuration keys for every build:

```
PYTHON_SERVICE_URL=http://127.0.0.1:<NORTH_A2A_PORT>
OLLAMA_ROUTER_URL=http://127.0.0.1:9999
AI_CORE_REGISTRY_URL=<EXISTING_PARADIGM_REGISTRY_URL>
```

8. Launch CALIAN.AI Angular Electron (architecture component 2)

Run the supplied Electron startup batch file if present. Alternatively, open Command Prompt in **CALIAN_AI_ELECTRON_THEMES** and run:

```
npm install     (first run only, if needed)
npm run dev
```

In the previously described development setup, Angular used port 4300 and the Node/Express bridge used port 8797; verify actual ports from the startup logs. The desktop window should show Agent Conversation, Intelligent Auto Select, Session Inspector, and the registry.

9. Verify seven-agent discovery

Confirm that the registry displays **7 agents total**: 1 local North Agent and 6 Paradigm/AI Core agents. If fewer appear, check the North service, RabbitMQ, Paradigm AI Service, and existing registry callbacks before routing tests.

10. Test Ollama agent selection (architecture component 3)

Select **Intelligent Auto Select**. Keep **Test auto-selection only (do not execute agent)** checked. Enter these prompts and inspect the routing result:

Prompt	Expected agent
Translate Good Morning into French	Translation Agent
Roll two dice	Dice Agent
Extract names, dates and addresses from this text	Structured Text Agent
Convert this speech recording into text	Speech Agent

Look for **Router: ollama** and the matching selected agent. Confidence may vary. The successful demonstration screenshot showed the Dice Agent selected via Ollama.

11. Execute a selected agent (separate verification)

Uncheck selection-only mode to request execution. Ensure the selected agent has a reachable execution endpoint and the application is configured to call it. Observe the streamed or returned response. **Successful selection alone does not establish successful execution.**

12. Verify Session Inspector

Field	Expected result
Router	ollama (not fallback)
Agent registry	7 total
Python/A2A	1 North Agent
Paradigm/AI Core	6 agents
Selected agent	Matches the request
Execution mode	Selection-only or enabled execution

13. Troubleshooting

Symptom	Check
Ollama unavailable	Ollama service on port 11434 and llama3.2 model
Python router unavailable	FastAPI process and port 9999
North Agent missing	Existing A2A process and configured endpoint
Paradigm agents missing	RabbitMQ, Micronaut AI Service, registry callbacks
Router shows fallback	Ollama connectivity and routing logs
Electron does not open	npm dependencies and startup console output
Selection works but execution fails	Agent endpoint and execution integration

14. Recommended startup sequence

1. Verify Ollama and llama3.2. 2. Verify RabbitMQ. 3. Start or verify Paradigm AI Service. 4. Start local North Agent. 5. Start Python Ollama router. 6. Launch Angular Electron. 7. Confirm seven agents. 8. Test translation and dice auto-selection. 9. Test execution separately.

15. Five-minute demo walkthrough

Minute 1: Introduce the Angular Electron workbench. **Minute 2:** Show one local North Agent and six Paradigm agents discovered via RabbitMQ. **Minute 3:** Auto-select Translation Agent. **Minute 4:** Auto-select Dice Agent and show Router: ollama. **Minute 5:** Return to Figure 1 and explain that Ollama selects an agent, while the selected agent performs the specialized task.